

Measurement Project

Introduction

ELO models are widely used, but have a key underlying assumption that the difference in latent skill/ability is the only thing that contributes to probability of winning. This inherently means that probabilities are transitive, i.e. $P(i > j)$ and $P(j > k) \Rightarrow P(i > k)$.

This works for a lot of games, such as chess, where the ELO model is widely used. Improvements to ELO are also popular taking into account Rating Deviation, the uncertainty of an individual's ability, estimated based on the consistency of results and the number of games played.

```
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```
Warning: package 'tibble' was built under R version 4.5.2
```

```
Warning: package 'tidyr' was built under R version 4.5.2
```

```
Warning: package 'purrr' was built under R version 4.5.2
```

```
Warning: package 'dplyr' was built under R version 4.5.2
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.1.5
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.2      v tibble     3.3.1
v lubridate  1.9.4      v tidyr      1.3.2
v purrr      1.2.1
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
x dplyr::lag()      masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become explicit
```

```
library(future.apply)
```

Warning: package 'future.apply' was built under R version 4.5.2

Loading required package: future

Warning: package 'future' was built under R version 4.5.2

```
library(dplyr)
library(ggplot2)
library(patchwork)
library(latex2exp)
```

Warning: package 'latex2exp' was built under R version 4.5.2

```
library(see)
```

Game simulation functions

```
simulate_elo_game <- function(thetas,N_opts=3){
  t_ids <- sample(c(1:length(thetas)),2,replace=FALSE)
  t <- thetas[t_ids]
  dt <- t[1]-t[2]
  p <- 1/(1+exp(-dt))
  w <- rbinom(n=1,size=1,prob=p)
  ## Hand played is randomly chosen
  h <- sample(c(1:N_opts),2,replace=TRUE)
  res <- c(t_ids[1],t_ids[2],h[1],h[2],w)
  return(res)
}

## Non-transitive payoff probability matrix (eps -> [0,2])
payoff_matrix <- function(N,eps=0) {
  eps <- abs(eps)
```

```

# generate square matrix with 0s

A <- matrix(0, nrow = N, ncol = N)

# Fill upper triangle with payoff
for(i in c(1:(N-1))){
  A[i,(i+1):N] <- rep(c(-1+eps, 1-eps), length.out = (N-i))
}

# Enforce skew-symmetry for cyclic game
A <- A - t(A)

# Convert to Probabilities
pA <- 0.5*(A + 1)
return(pA)
}

simulate_rps <- function(thetas,N_opts,payoff=NULL,replace=FALSE){
  t_ids <- sample(c(1:length(thetas)),2,replace=FALSE)
  if(is.null(payoff)){
    p <- payoff_matrix(N_opts,eps=0)
  }else{
    p <- payoff
  }

  h <- sample(c(1:N_opts),2,replace=replace)
  w <- rbinom(1,1,p[h[1],h[2]])
  # w <- ifelse(prob>0,rbinom(1,1,prob),rbinom(1,1,1+prob))
  res <- c(t_ids[1],t_ids[2],h[1],h[2],w)
  return(res)
}

simulate_rps_luck <- function(thetas,N_opts){
  t_ids <- sample(c(1:length(thetas)),2,replace=FALSE)

  p <- payoff_matrix(N_opts)
  idx <- which(p>0.5,arr.ind = TRUE,useNames = FALSE)
  rps_wins <- list()
  for (i in c(1:nrow(idx))){
    rps_wins[[i]] <- c(idx[i,1],idx[i,2])
  }
}

```

```

h <- sample(rps_wins,1)[[1]]

## Luckier player to win
dt <- thetas[t_ids[1]] - thetas[t_ids[2]]
w <- rbinom(1,1,plogis(dt))
if(w==0){
  h <- c(h[2],h[1])
}
res <- c(t_ids[1],t_ids[2],h[1],h[2],w)
return(res)
}

simulate_NT_model <- function(thetas,phis,N_opts,replace=FALSE){
  t_ids <- sample(c(1:length(thetas)),2,replace=FALSE)
  phi_ids <- sample(c(1:length(phis)),2,replace=FALSE)
  h <- sample(c(1:N_opts),2,replace=replace)

  d_theta <- thetas[t_ids[1]] - thetas[t_ids[2]]
  d_phi <- phis[h[1]] - phis[h[2]]
  d_phi <- atan2(sin(d_phi),cos(d_phi))
  g_d_phi <- sinpi(d_phi)
  # g_d_phi <- 2*atanh(sinpi(d_phi))

  logit <- d_theta + g_d_phi

  p <- plogis(logit)
  w <- rbinom(1,1,p)

  res <- c(t_ids[1],t_ids[2],h[1],h[2],w)
  return(res)
}

```

Estimation functions

```

LogLikELO <- function(data, theta) {
  p <- plogis(theta[data$player1] - theta[data$player2])
  ll <- sum(data$result * log(p) + (1 - data$result) * log(1 - p))
  return(ll)
}

```

```

fit_elo_model <- function(N_players,matches){

  starting_theta <- rnorm(N_players)

  sol <- optim(
    starting_theta,
    LogLikELO,
    data = matches,
    method = 'L-BFGS-B',
    lower = -3,
    upper = 3,
    control = list(fnscale = -1, maxit = 1e6)
  )

  return(sol)
}

## Model with non-transitive effects (additive)
LogLik_RPS <- function(params, data, N_ids) {
  theta <- c(0,params[1:(N_ids-1)])
  phi <- c(0,params[(N_ids):length(params)])
  d_theta <- theta[data$player1] - theta[data$player2]

  d_phi <- phi[data$hand1] - phi[data$hand2]
  d_phi <- atan2(sin(d_phi),cos(d_phi))
  ## Non-linear function of delta Phi -> g(delta_phi)
  g_d_phi <- sinpi(d_phi)

  logit <- d_theta+g_d_phi
  # logit <- d_theta+2*atanh(sinpi(d_phi))

  p <- plogis(logit)
  ll <- sum(data$result * log(p) + (1 - data$result) * log(1 - p))

  ## Ridge penalty
  lambda <- 0.1
  # penalty <- lambda*sum(abs(params)) ## Lasso
  penalty <- lambda*sum(theta^2)

  neg_ll <- -ll + penalty

  ##

```

```

if(is.finite(neg_ll)){
  return(neg_ll)
}else{
  return(1e10)
}
}

estimate_nt_elo <- function(N_ids,N_opts,matches,n_start=5){

  opt_sol <- NA
  opt_sol_val <- Inf
  opt_start <- NA
  for (k in c(1:(n_start+2))){
    # print(paste0("Starting conditions: ",k))
    starting_theta <- rnorm(N_ids-1)
    if (k == 1){
      starting_phi <- rep(0.25,N_opts-1)
    }else if (k == 2){
      starting_phi <- c(1:(N_opts-1))*2/N_opts
    }else {
      starting_phi <- runif(N_opts-1,min=0,max=2)
    }

    starting_params <- c(starting_theta,starting_phi)
    sol <- optim(
      starting_params,
      LogLik_RPS,
      data = matches,
      N_ids = N_ids,
      method = 'L-BFGS-B',
      hessian = TRUE,
      lower = c(rep(-Inf,(N_ids-1)),rep(0,(N_opts-1))),
      upper = c(rep(Inf,(N_ids-1)),rep(2*pi,(N_opts-1))),
      control = list(fnscale = 1,
                     maxit = 1e6
                    ))
    if(sol$value < opt_sol_val){
      opt_sol_val <- sol$value
      opt_sol <- sol
      opt_start <- starting_phi
    }
  }
}

```

```

}

return(list(sol=opt_sol,starting_phi=opt_start))
}

## Model with Beta factor and other experiments
m_factor <- function(dt,dp){
  return(sign(-sign(dt)*sign(dp) + (sign(dt)+sign(dp))))
}

LogLik_RPS_experiemnt <- function(params, data, N_ids, N_opts) {
  theta <- c(0,params[1:(N_ids-1)])
  phi <- c(0,params[(N_ids):(N_ids + N_opts - 2)])
  beta <- tail(params,1)
  d_theta <- theta[data$player1] - theta[data$player2]

  d_phi <- phi[data$hand1] - phi[data$hand2]
  d_phi <- atan2(sin(d_phi),cos(d_phi))

  ## Non-linear function of delta Phi -> g(delta_phi)
  # g_d_phi <- beta*sinpi(d_phi)
  g_d_phi <- 2*atanh(sinpi(d_phi))
  # g_d_phi <- sinpi(d_phi)
  # g_d_phi <- 2*atanh(sinpi(d_phi))

  logit <- d_theta + g_d_phi + 0*sign(g_d_phi)*d_theta*g_d_phi
  # logit <- (1+d_theta)*(1+g_d_phi)-1
  # d_r <- r[data$hand2]/r[data$hand1]

  p <- plogis(logit)
  ll <- sum(data$result * log(p) + (1 - data$result) * log(1 - p))

  ## Ridge penalty
  lambda <- 0.1
  # penalty <- lambda*sum(abs(params))
  penalty <- lambda*sum(theta^2)

  neg_ll <- -ll + penalty
  if(is.finite(neg_ll)){
    return(neg_ll)
  }
}

```

```

}else{
  return(1e10)
}
# return(neg_ll)
}

estimate_nt_elo_experiemnt <- function(N_ids,N_opts,matches,n_start=5){

  # starting_phi <- rep(0.25,N_opts-1)

  # starting_phi <- c(1:(N_opts-1))*2/N_opts + rnorm((N_opts-1),0,0.1)

  # starting_phi <- (c(1:N_opts)-1)*2/N_opts
  # starting_phi <- sample(starting_phi,N_opts,FALSE)
  opt_sol <- NA
  opt_sol_val <- Inf
  opt_start <- NA
  for (k in c(1:(n_start+2))){
    starting_theta <- rnorm(N_ids-1)
    starting_beta <- 1
    if (k == 1){
      starting_phi <- rep(0.25,N_opts-1)
    }else if (k == 2){
      starting_phi <- c(1:(N_opts-1))*2*pi/N_opts
    }else {
      starting_phi <- runif(N_opts-1,min=0,max=2)
    }

    starting_params <- c(starting_theta,starting_phi,starting_beta)
    sol <- optim(
      starting_params,
      LogLik_RPS_experiemnt,
      data = matches,
      N_ids = N_ids,
      N_opts = N_opts,
      method = 'L-BFGS-B',
      hessian = TRUE,
      lower = c(rep(-Inf,(N_ids-1)),rep(0,(N_opts-1)),0),
      upper = c(rep(Inf,(N_ids-1)),rep(2*pi,(N_opts-1)),1),
      control = list(fnscale = 1,
                     maxit = 1e6
                    ))
  }
}

```



```

    if(sol$value < opt_sol_val){
      opt_sol_val <- sol$value
      opt_sol <- sol
      opt_start <- starting_phi
    }
  }

  return(list(sol=opt_sol,starting_phi=opt_start))
}

```

Plotting functions

```

plot_elo <- function(sol,thetas){
  df <- data.frame(theta=thetas,theta_est=sol$par)
  p <- ggplot(df,aes(x=theta,y=theta_est)) +
    geom_point(color=okabeito_colors(3)) +
    theme_bw() +
    labs(x=TeX("$\\theta$"),y=TeX("$\\hat{\\theta}$")) +
    annotate("text", x = 2, y = -3,
             label = paste("r = ", round(cor(df$theta,df$theta_est,method="spearman"),2)),
             size=4) + ggtitle("True vs. Estimated thetas (ELO Model)")
  return(p)
}

plot_nt_elo <- function(sol,thetas,N_ids,N_opts,labels=c("Rock", "Paper", "Scissors")){

  est_theta <- as.data.frame(list(est=c(0,sol$par[1:(N_ids-1)]),
                                true=thetas))
  est_phi <- as.data.frame(list(est=c(0,sol$par[(N_ids):(N_ids+N_opts-2)]),
                                r=rep(1,N_opts),
                                type=rep("est",N_opts),
                                id=c(1:N_opts)))

  est_phi <- est_phi |> mutate(phi=pi*(est%%2))
  # est_phi <- est_phi |> mutate(phi=(est%%2))

  est_theta_mu <- round(mean(est_theta$est),2)
  est_theta_var <- round(sd(est_theta$est)^2,3)
}

```

```

p_phi <- ggplot(est_phi) +
  geom_point(aes(r, phi,color=factor(id)),size=2) +
  coord_radial(theta="y",
               start=pi/2,
               direction=-1,
               rlim=c(0,1),
               thetalim = c(0,2*pi)) +
  ggtitle("Phi estimates") +
  labs(color = "Hand",shape="") +
  scale_color_discrete(labels = labels)

p_theta <- ggplot(est_theta,aes(x=true,y=est)) +
  geom_point() +
  theme_bw() +
  ggtitle(paste0("Theta ~ N(",est_theta_mu,",",est_theta_var,")", r=",
               round(cor(est_theta$est,est_theta$true,method="spearman"),2)))

return(p_theta | p_phi)
}

```

Simulating Games and Testing model fits

ELO Data

```

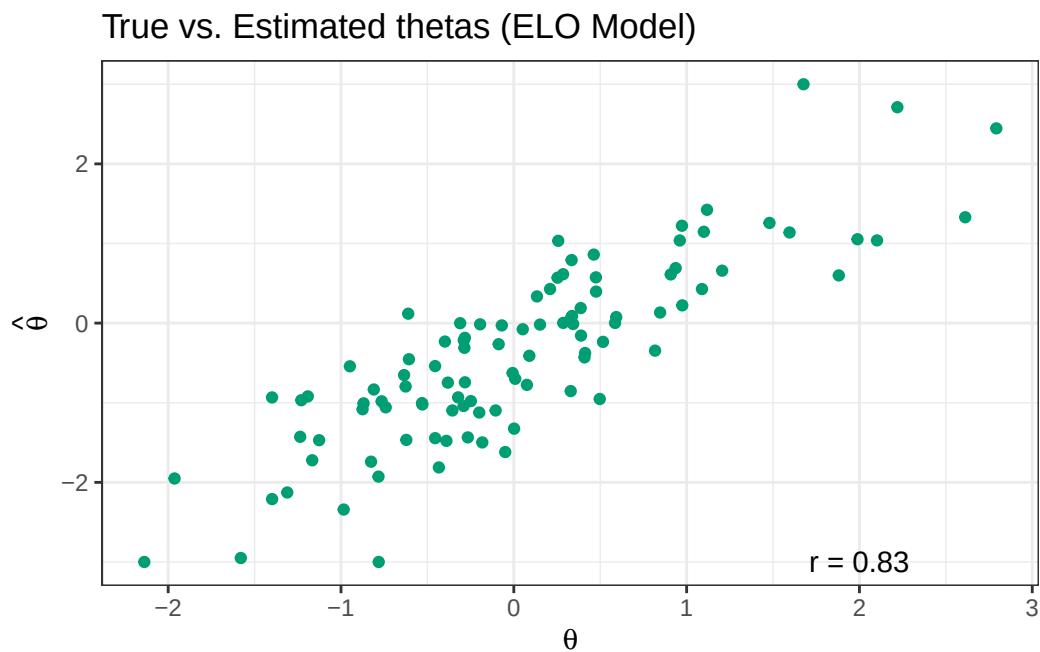
## Simulating elo game data
N_ids <- 100
N_games <- 1000
elo_thetas <- rnorm(N_ids)

elo_matches <- as.data.frame(do.call(rbind, replicate(N_games,simulate_elo_game(elo_thetas),
                                                    simplify=FALSE)))

elo_matches <- elo_matches |> rename(player1="V1",
                                   player2="V2",
                                   hand1 = "V3",
                                   hand2 = "V4",
                                   result="V5")

```

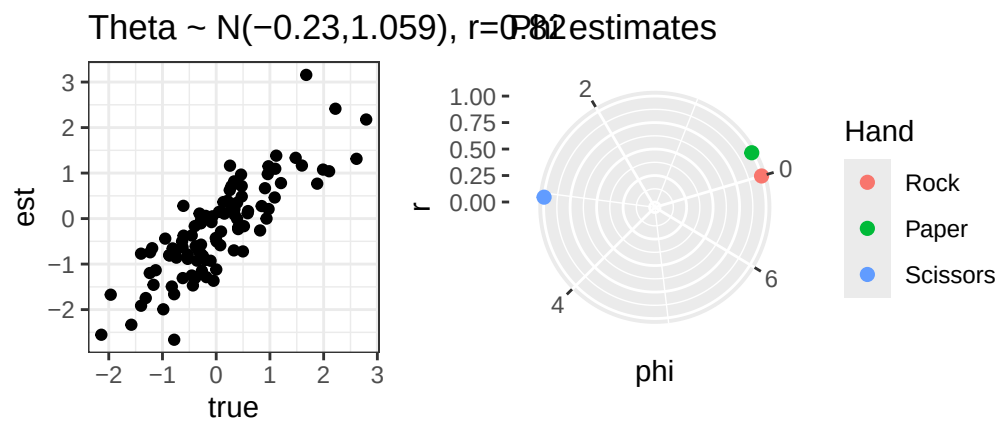
```
elo_sol <- fit_elo_model(N_ids,elo_matches)
plot_elo(elo_sol,elo_thetas)
```



```
res_nt <- estimate_nt_elo(N_ids,N_opts=3,elo_matches)
plot_nt_elo(res_nt$sol,elo_thetas,N_ids,N_opts=3)
```

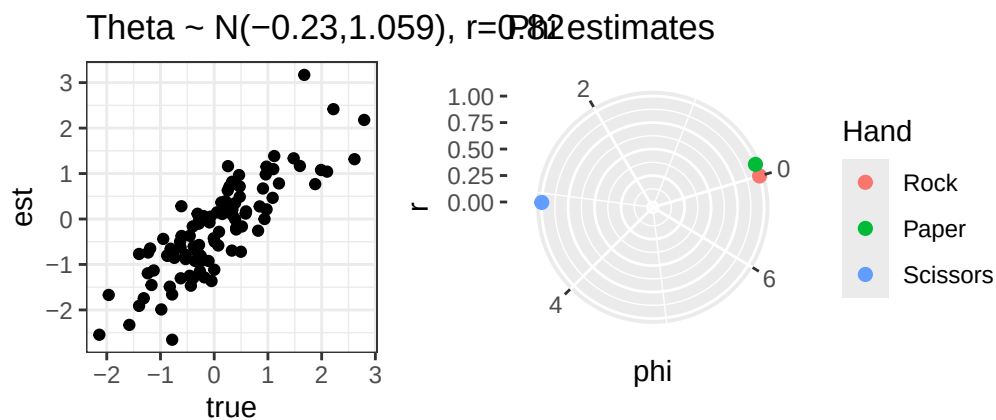
Warning: The `direction` argument of `coord\_radial()` is deprecated as of ggplot2 4.0.0.
i Please use the `reverse` argument instead.

Ignoring unknown labels:
* shape : ""



```
res_nt <- estimate_nt_elo_experiemment(N_ids,N_opts=3,elo_matches)
plot_nt_elo(res_nt$sol,elo_thetas,N_ids,N_opts=3)
```

Ignoring unknown labels:
 * shape : ""



Weird outliers in theta estimation make me believe that some ridge penalty on theta would be good to have. Getting Reasonable estimates from the non-transitive models, d_phis are small (or close to π), but not exactly 0 or π (what I would expect)

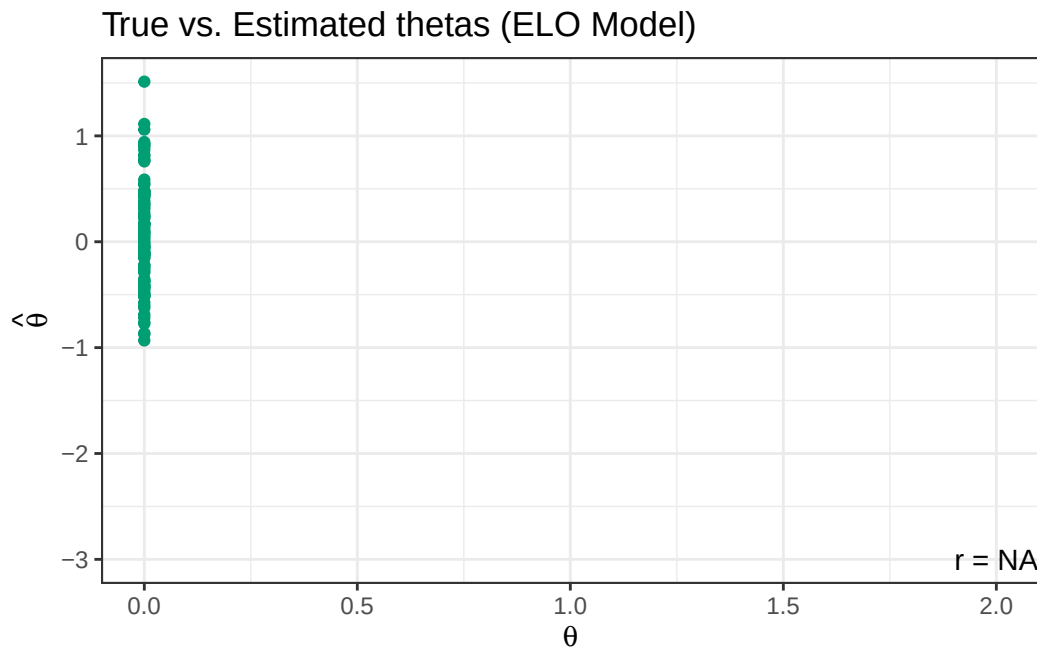
Rock-Paper-Scissors

```
## Simulating the game data
set.seed(1729)
N_ids <- 100
N_games <- 1000
N_opts <- 3 ## For rock-paper-scissors
rps_thetas <- rep(0,N_ids)

rps_matches <- as.data.frame(do.call(rbind, replicate(N_games,simulate_rps(rps_thetas,N_opts,
rps_matches <- rps_matches |> rename(player1="V1",
                                     player2="V2",
                                     hand1 = "V3",
                                     hand2 = "V4",
                                     result="V5")
```

```
## Fitting ELO model to RPS Data
rps_elo_sol <- fit_elo_model(N_ids,rps_matches)
plot_elo(rps_elo_sol,rps_thetas)
```

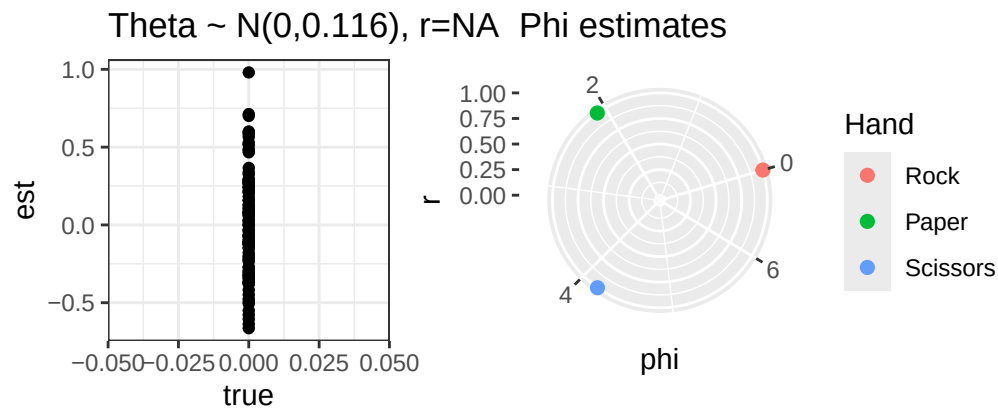
Warning in cor(df\$theta, df\$theta_est, method = "spearman"): the standard deviation is zero



```
## Fitting simple non-transitive model
res_nt <- estimate_nt_elo(N_ids, N_opts, rps_matches)
plot_nt_elo(res_nt$sol,rps_thetas,N_ids,N_opts,labels=c("Rock", "Paper", "Scissors","Spock",
```

Warning in cor(est_theta\$est, est_theta\$true, method = "spearman"): the standard deviation is zero

Ignoring unknown labels:
* shape : ""

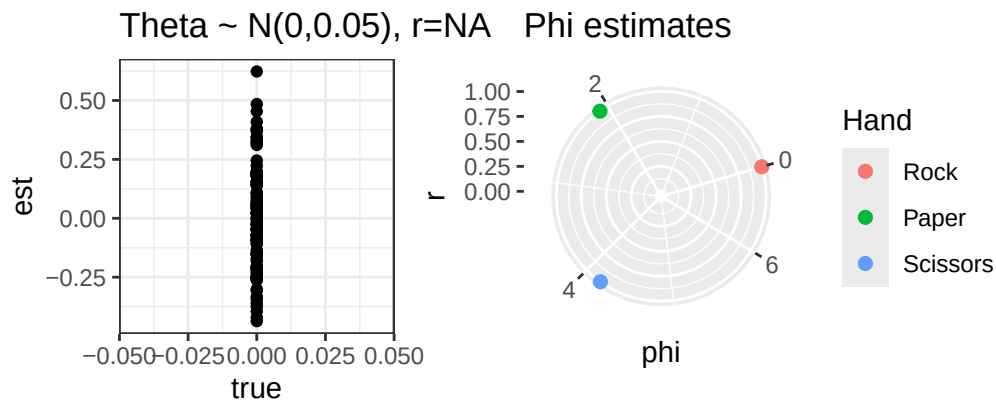


```
## Fitting experiemental non-transitive model
res_nt <- estimate_nt_elo_experiemment(N_ids, N_opts, rps_matches)
plot_nt_elo(res_nt$sol,rps_thetas,N_ids,N_opts,labels=c("Rock", "Paper", "Scissors","Spock",
```

Warning in cor(est_theta\$est, est_theta\$true, method = "spearman"): the standard deviation is zero

Ignoring unknown labels:

* shape : ""



```
## Simulating the game data
set.seed(1729)
N_ids <- 100
N_games <- 1000
N_opts <- 5 ## For rock-paper-scissors
rps_thetas <- rep(0, N_ids)

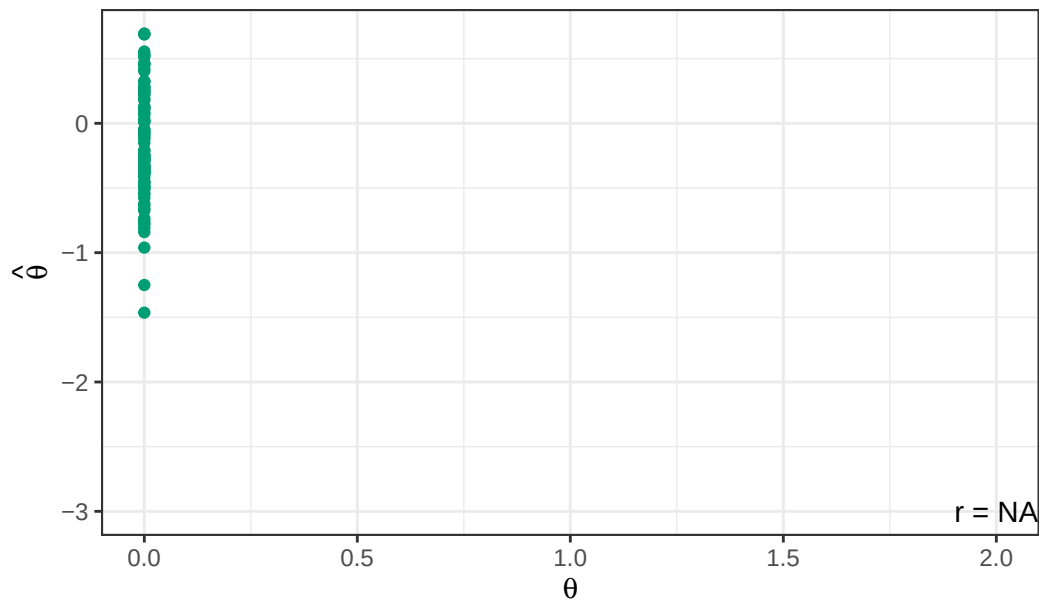
rps_matches <- as.data.frame(do.call(rbind, replicate(N_games, simulate_rps(rps_thetas, N_opts))))

rps_matches <- rps_matches |> rename(player1="V1",
                                     player2="V2",
                                     hand1 = "V3",
                                     hand2 = "V4",
                                     result="V5")

## Fitting ELO model to RPS Data
rps_elo_sol <- fit_elo_model(N_ids, rps_matches)
plot_elo(rps_elo_sol, rps_thetas)
```

Warning in cor(df\$theta, df\$theta_est, method = "spearman"): the standard deviation is zero

True vs. Estimated thetas (ELO Model)

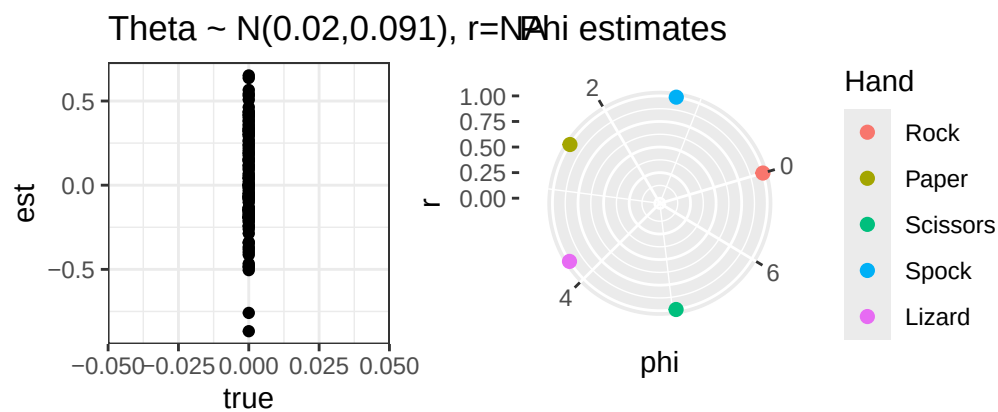


```
## Fitting simple non-transitive model
res_nt <- estimate_nt_elo(N_ids, N_opts, rps_matches)
plot_nt_elo(res_nt$sol,rps_thetas,N_ids,N_opts,labels=c("Rock", "Paper", "Scissors","Spock",
```

Warning in cor(est_theta\$est, est_theta\$true, method = "spearman"): the standard deviation is zero

Ignoring unknown labels:

* shape : ""

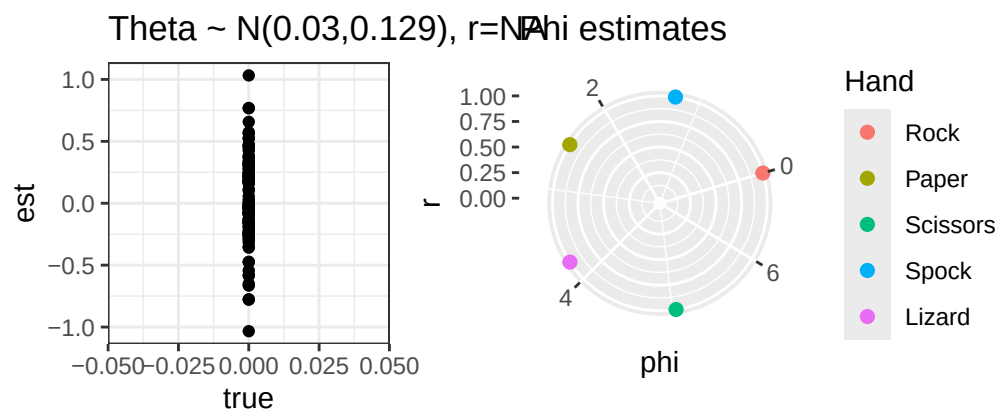


```
## Fitting experiemental non-transitive model
res_nt <- estimate_nt_elo_experiemment(N_ids, N_opts, rps_matches)
plot_nt_elo(res_nt$sol,rps_thetas,N_ids,N_opts,labels=c("Rock", "Paper", "Scissors","Spock",
```

Warning in cor(est_theta\$est, est_theta\$true, method = "spearman"): the standard deviation is zero

Ignoring unknown labels:

* shape : ""



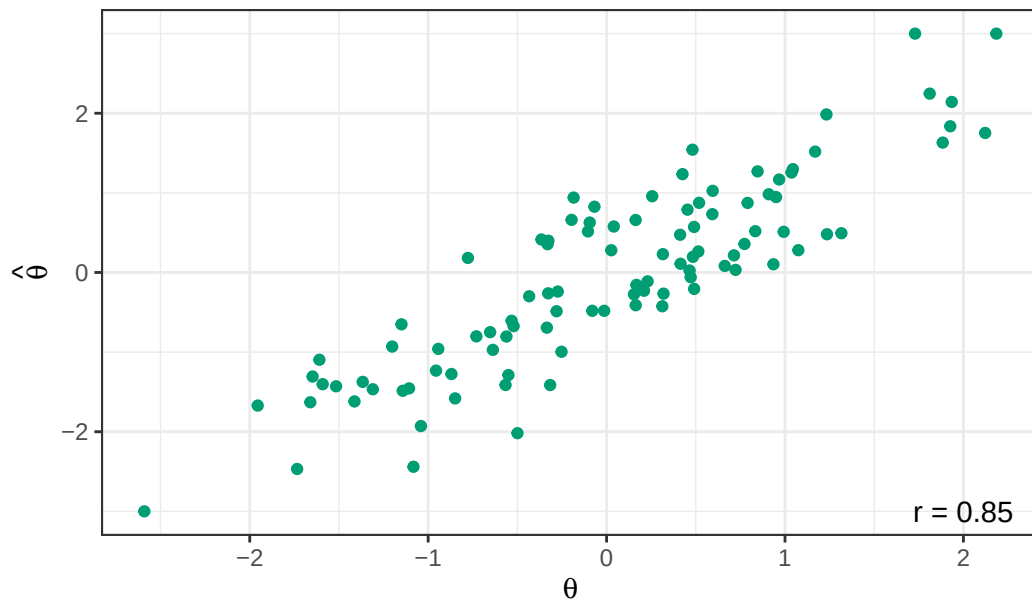
RPS with “luck”

```
N_ids <- 100
N_opts <- 5
N_games <- 1000
rps_luck_thetas <- rnorm(N_ids,mean=0,sd=1)
rps_luck_matches <- as.data.frame(do.call(rbind, replicate(N_games,simulate_rps_luck(rps_luck_thetas))))

rps_luck_matches <- rps_luck_matches |> rename(player1="V1",
                                              player2="V2",
                                              hand1 = "V3",
                                              hand2 = "V4",
                                              result="V5")
```

```
## Fitting an ELO Model to luck data
res_elo_rps_luck <- fit_elo_model(N_ids,rps_luck_matches)
plot_elo(res_elo_rps_luck,rps_luck_thetas)
```

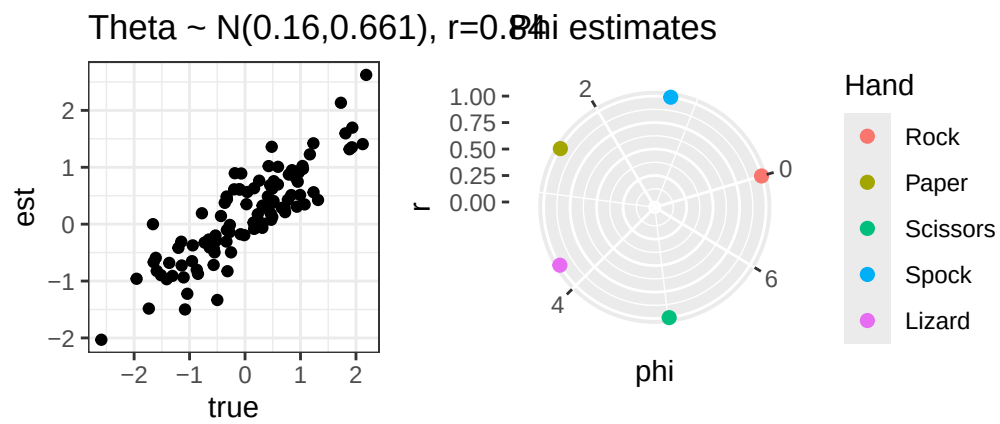
True vs. Estimated thetas (ELO Model)



```
res_rps_luck <- estimate_nt_elo(N_ids, N_opts, rps_luck_matches, 10)
plot_nt_elo(res_rps_luck$sol, rps_luck_thetas, N_ids, N_opts, labels=c("Rock", "Paper", "Scissors"))
```

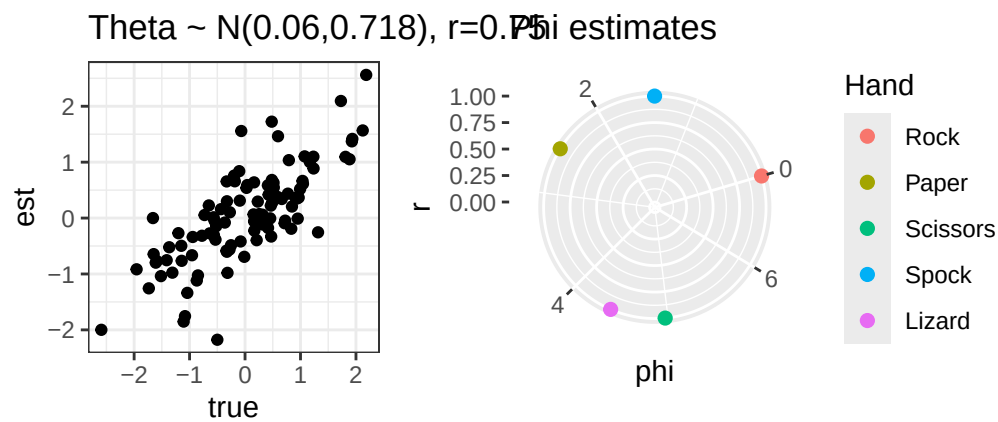
Ignoring unknown labels:

* shape : ""



```
res_rps_luck_exp <- estimate_nt_elo_experiemment(N_ids, N_opts, rps_luck_matches,10)
plot_nt_elo(res_rps_luck_exp$sol,rps_luck_thetas,N_ids,N_opts,labels=c("Rock", "Paper", "Sci
```

```
Ignoring unknown labels:
* shape : ""
```



Non-Transitive Dice game

```
grime_3 <- c(7/12,11/36,7/12) #Red, Blue, Olive
grime_4 <- c(2/3,1/3,4/9,1/2,2/3,1/3) #Red, Blue, Olive, Yellow
grime_5 <- c(7/12,11/36,13/18,4/9,7/12,1/3,2/3,5/9,5/18,5/9) #Red, Blue, Olive, Yellow, Mager

grime_mats_list <- list(grime_3,grime_4,grime_5)

grime_dice_payoffs <- function(k,grime_mats=grime_mats_list){
  p <- matrix(0,k,k)
  upper_tri_idx <- as.data.frame(which(upper.tri(p),arr.ind = TRUE)) |>
    arrange(row,col) |>
    as.matrix()

  lower_tri_idx <- as.data.frame(which(lower.tri(p),arr.ind = TRUE)) |>
    arrange(col,row) |>
    as.matrix()

  p[upper_tri_idx] <- grime_mats[[k-2]]
  p[lower_tri_idx] <- 1-p[upper_tri_idx]
```

```
diag(p) <- 1/2
return(p)
}
```

3-dice

```
N_games <- 1000
N_ids <- 100
N_opts <- 3
thetas <- rnorm(N_ids)

payoff_mat <- grime_dice_payoffs(N_opts)
grime_dice_matches <- as.data.frame(
  do.call(rbind,
    replicate(N_games,
      simulate_rps(thetas, N_opts, payoff_mat, TRUE),
      simplify = FALSE)))

grime_dice_matches <- grime_dice_matches |> rename(player1="V1",
  player2="V2",
  hand1 = "V3",
  hand2 = "V4",
  result="V5")
```

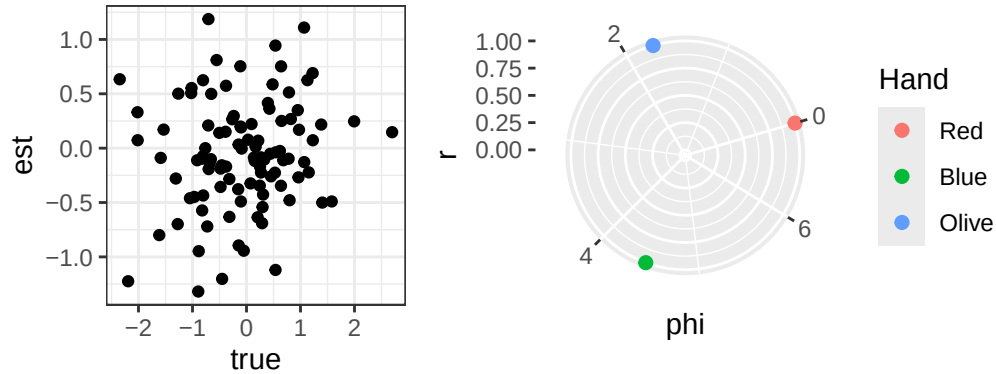
```
payoff_mat
```

```
      [,1]      [,2]      [,3]
[1,] 0.5000000 0.5833333 0.3055556
[2,] 0.4166667 0.5000000 0.5833333
[3,] 0.6944444 0.4166667 0.5000000
```

```
res <- estimate_nt_elo(N_ids, N_opts, grime_dice_matches, n_start=10)
plot_nt_elo(res$sol, thetas, N_ids, N_opts, labels=c("Red", "Blue", "Olive", "Yellow", "Magenta"))
```

```
Ignoring unknown labels:
* shape : ""
```

Theta ~ N(-0.06,0.253), r=0.413 estimates



```
est_thetas <- c(0,res$sol$par[1:(N_ids-1)])
est_phis <- c(0,tail(res$sol$par,N_opts-1))
post_estimation <- grime_dice_matches |>
  mutate(d_theta = est_thetas[player1]-est_thetas[player2],
         d_phi = est_phis[hand1]-est_phis[hand2],
         d_phi2 = atan2(sin(d_phi),cos(d_phi)),
         exponent = -(d_theta+sinpi(d_phi2)),
         prob = 1/(1+exp(exponent))) |>
  group_by(hand1,hand2) |>
  summarise(expected_payoff=mean(prob),.groups = "drop_last")

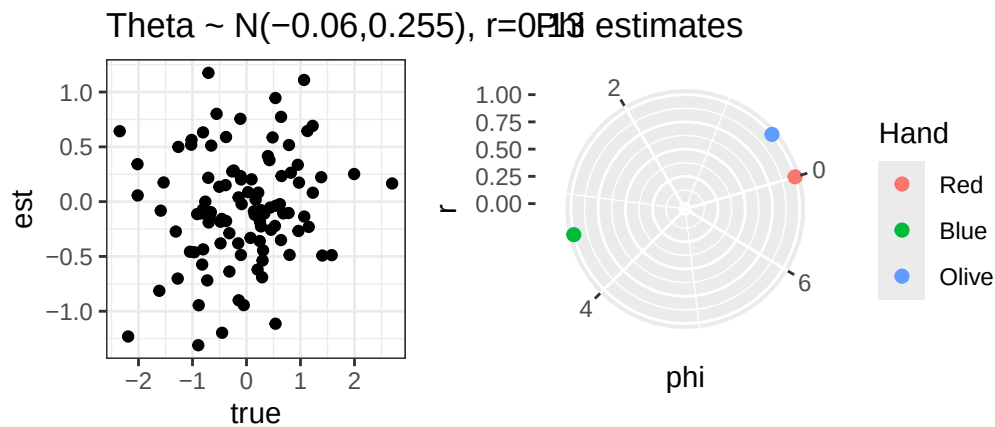
est_payoff_mat <- xtabs(expected_payoff ~ hand1 + hand2, data=post_estimation)
est_payoff_mat
```

	hand2		
hand1	1	2	3
1	0.5291493	0.5782343	0.2850961
2	0.3908591	0.5115872	0.5949157
3	0.7065241	0.4278879	0.5144078


```
res <- estimate_nt_elo_experience(N_ids,N_opts,grime_dice_matches,n_start=10)
plot_nt_elo(res$sol,thetas,N_ids,N_opts,labels=c("Red", "Blue", "Olive", "Yellow", "Magenta"))
```

Ignoring unknown labels:

```
* shape : ""
```



```
est_thetas <- c(0,res$sol$par[1:(N_ids-1)])
est_phis <- c(0,tail(res$sol$par,N_opts-1))
post_estimation <- grime_dice_matches |>
  mutate(d_theta = est_thetas[player1]-est_thetas[player2],
         d_phi = est_phis[hand1]-est_phis[hand2],
         d_phi2 = atan2(sin(d_phi),cos(d_phi)),
         exponent = -(d_theta+2*atanh(sinpi(d_phi2))),
         prob = 1/(1+exp(exponent))) |>
  group_by(hand1,hand2) |>
  summarise(expected_payoff=mean(prob),.groups = "drop_last")

est_payoff_mat <- xtabs(expected_payoff ~ hand1 + hand2, data=post_estimation)
est_payoff_mat
```

hand2

hand1	1	2	3
1	0.5291238	0.2742276	0.4938928
2	0.6999295	0.5113551	0.2997242
3	0.4981270	0.7222777	0.5142989

4-dice

```
N_games <- 1000
N_ids <- 100
N_opts <- 4
thetas <- rnorm(N_ids)

payoff_mat <- grime_dice_payoffs(N_opts)
grime_dice_matches <- as.data.frame(
  do.call(rbind,
    replicate(N_games,
      simulate_rps(thetas, N_opts, payoff_mat, TRUE),
      simplify = FALSE)))

grime_dice_matches <- grime_dice_matches |> rename(player1="V1",
  player2="V2",
  hand1 = "V3",
  hand2 = "V4",
  result="V5")
```

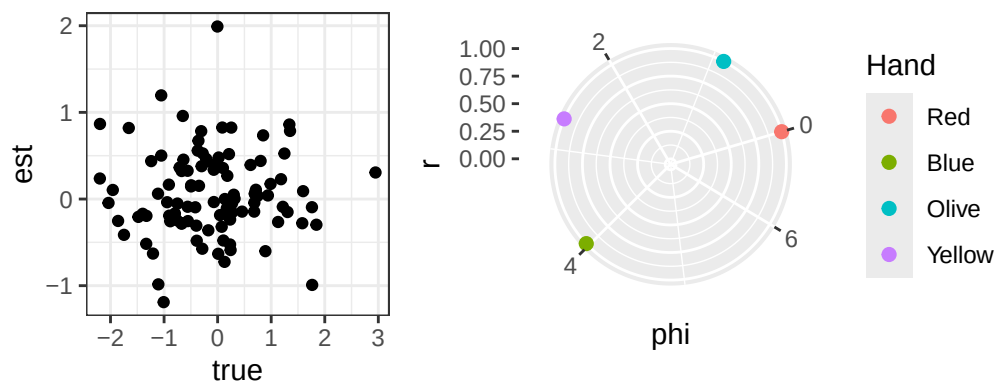
```
payoff_mat
```

	[,1]	[,2]	[,3]	[,4]
[1,]	0.5000000	0.6666667	0.3333333	0.4444444
[2,]	0.3333333	0.5000000	0.5000000	0.6666667
[3,]	0.6666667	0.5000000	0.5000000	0.3333333
[4,]	0.5555556	0.3333333	0.6666667	0.5000000

```
res <- estimate_nt_elo(N_ids, N_opts, grime_dice_matches, n_start=10)
plot_nt_elo(res$sol, thetas, N_ids, N_opts, labels=c("Red", "Blue", "Olive", "Yellow", "Magenta"))
```

```
Ignoring unknown labels:
* shape : ""
```

Theta ~ N(0.06,0.243), $r=0.074$ phi estimates



```
est_thetas <- c(0,res$sol$par[1:(N_ids-1)])
est_phis <- c(0,tail(res$sol$par,N_opts-1))
post_estimation <- grime_dice_matches |>
  mutate(d_theta = est_thetas[player1]-est_thetas[player2],
         d_phi = est_phis[hand1]-est_phis[hand2],
         d_phi2 = atan2(sin(d_phi),cos(d_phi)),
         exponent = -(d_theta+sinpi(d_phi2)),
         prob = 1/(1+exp(exponent))) |>
  group_by(hand1,hand2) |>
  summarise(expected_payoff=mean(prob),.groups = "drop_last")

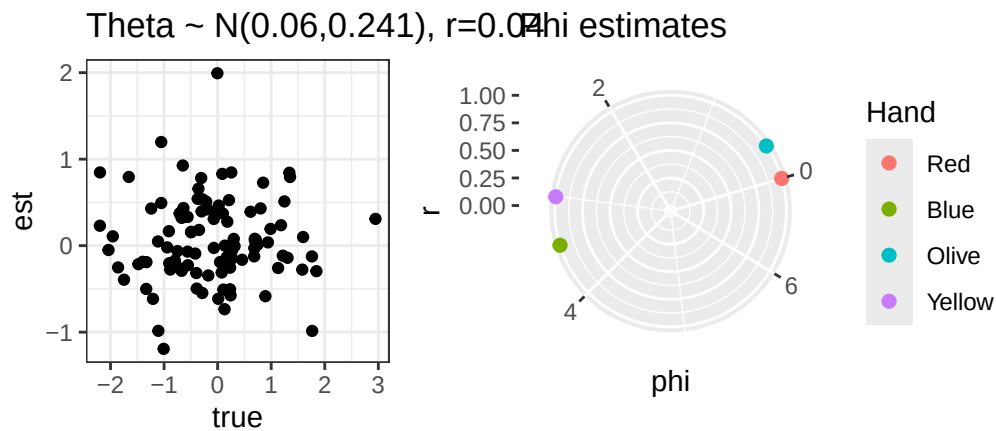
est_payoff_mat <- xtabs(expected_payoff ~ hand1 + hand2, data=post_estimation)
est_payoff_mat
```

	hand2			
hand1	1	2	3	4
1	0.4704571	0.6924448	0.3423426	0.4209603
2	0.3626215	0.4980358	0.4937843	0.7071369
3	0.6704499	0.5060750	0.5369281	0.2982401
4	0.6181983	0.2932520	0.6959850	0.4739636

```
res <- estimate_nt_elo_experiemment(N_ids,N_opts,grime_dice_matches,n_start=10)
plot_nt_elo(res$sol,thetas,N_ids,N_opts,labels=c("Red", "Blue", "Olive", "Yellow", "Magenta"))
```

Ignoring unknown labels:

```
* shape : ""
```



```
est_thetas <- c(0,res$sol$par[1:(N_ids-1)])
est_phis <- c(0,tail(res$sol$par,N_opts-1))
post_estimation <- grime_dice_matches |>
  mutate(d_theta = est_thetas[player1]-est_thetas[player2],
         d_phi = est_phis[hand1]-est_phis[hand2],
         d_phi2 = atan2(sin(d_phi),cos(d_phi)),
         exponent = -(d_theta+2*atanh(sinpi(d_phi2))),
         prob = 1/(1+exp(exponent))) |>
  group_by(hand1,hand2) |>
  summarise(expected_payoff=mean(prob),.groups = "drop_last")

est_payoff_mat <- xtabs(expected_payoff ~ hand1 + hand2, data=post_estimation)
est_payoff_mat
```

hand2

hand1	1	2	3	4
1	0.4709223	0.3731140	0.4509949	0.5150151
2	0.6819134	0.4976091	0.2654624	0.3499132
3	0.5630243	0.7294828	0.5364927	0.4455443
4	0.5268887	0.6609468	0.5466894	0.4737771

5-dice

```
N_games <- 1000
N_ids <- 100
N_opts <- 5
thetas <- rnorm(N_ids)

payoff_mat <- grime_dice_payoffs(N_opts)
grime_dice_matches <- as.data.frame(
  do.call(rbind,
    replicate(N_games,
      simulate_rps(thetas, N_opts, payoff_mat, TRUE),
      simplify = FALSE)))

grime_dice_matches <- grime_dice_matches |> rename(player1="V1",
  player2="V2",
  hand1 = "V3",
  hand2 = "V4",
  result="V5")
```

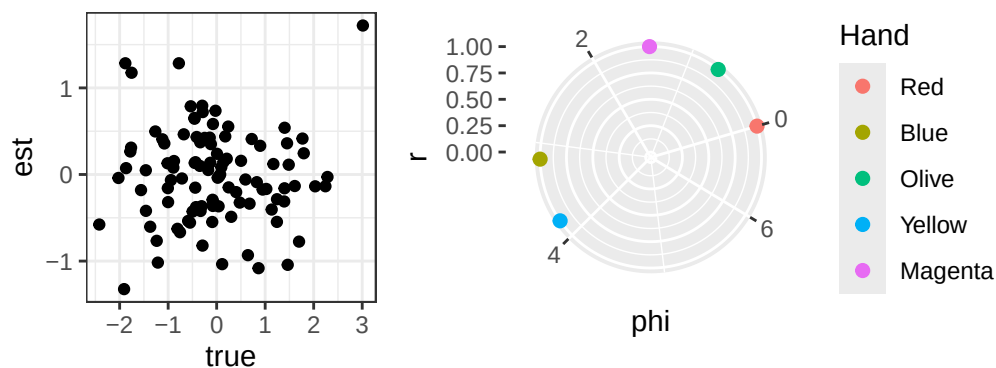
payoff_mat

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0.5000000	0.5833333	0.3055556	0.7222222	0.4444444
[2,]	0.4166667	0.5000000	0.5833333	0.3333333	0.6666667
[3,]	0.6944444	0.4166667	0.5000000	0.5555556	0.2777778
[4,]	0.2777778	0.6666667	0.4444444	0.5000000	0.5555556
[5,]	0.5555556	0.3333333	0.7222222	0.4444444	0.5000000

```
res <- estimate_nt_elo(N_ids, N_opts, grime_dice_matches, n_start=10)
plot_nt_elo(res$sol, thetas, N_ids, N_opts, labels=c("Red", "Blue", "Olive", "Yellow", "Magenta"))
```

Ignoring unknown labels:
 * shape : ""

Theta ~ N(-0.01,0.289), r=Phi04estimates



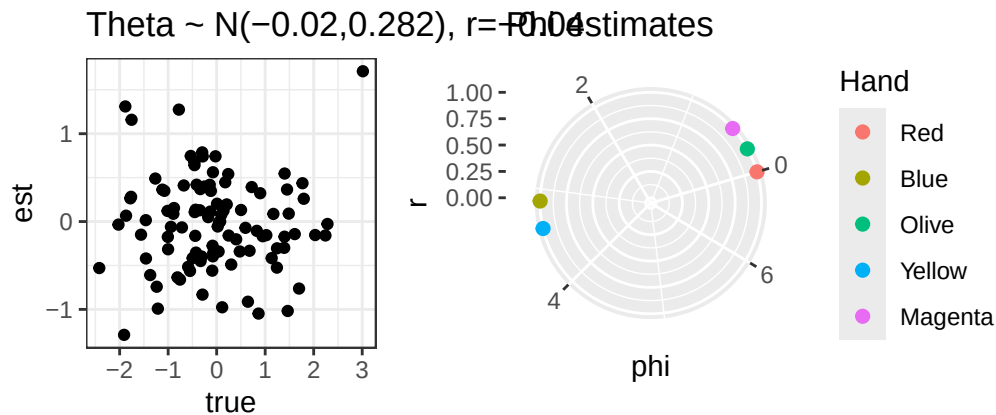
```
est_thetas <- c(0,res$sol$par[1:(N_ids-1)])
est_phis <- c(0,tail(res$sol$par,N_opts-1))
post_estimation <- grime_dice_matches |>
  mutate(d_theta = est_thetas[player1]-est_thetas[player2],
         d_phi = est_phis[hand1]-est_phis[hand2],
         d_phi2 = atan2(sin(d_phi),cos(d_phi)),
         exponent = -(d_theta+sinpi(d_phi2)),
         prob = 1/(1+exp(exponent))) |>
  group_by(hand1,hand2) |>
  summarise(expected_payoff=mean(prob),.groups = "drop_last")

est_payoff_mat <- xtabs(expected_payoff ~ hand1 + hand2, data=post_estimation)
est_payoff_mat
```

	hand2					
hand1	1	2	3	4	5	
1	0.5095325	0.4989417	0.3610782	0.6415078	0.2978266	
2	0.5200201	0.5597161	0.6650910	0.3396979	0.7319646	
3	0.6703610	0.3629082	0.5005043	0.5153700	0.3363908	
4	0.3982688	0.6237937	0.4971506	0.4856101	0.6475392	
5	0.7222766	0.2784272	0.6475964	0.3822104	0.4879762	

```
res <- estimate_nt_elo_experiment(N_ids,N_opts,grime_dice_matches,n_start=10)
plot_nt_elo(res$sol,thetas,N_ids,N_opts,labels=c("Red", "Blue", "Olive", "Yellow", "Magenta"))
```

Ignoring unknown labels:
 * shape : ""



```
est_thetas <- c(0,res$sol$par[1:(N_ids-1)])
est_phis <- c(0,tail(res$sol$par,N_opts-1))
post_estimation <- grime_dice_matches |>
  mutate(d_theta = est_thetas[player1]-est_thetas[player2],
         d_phi = est_phis[hand1]-est_phis[hand2],
         d_phi2 = atan2(sin(d_phi),cos(d_phi)),
         exponent = -(d_theta+2*atanh(sinpi(d_phi2))),
         prob = 1/(1+exp(exponent))) |>
  group_by(hand1,hand2) |>
  summarise(expected_payoff=mean(prob),.groups = "drop_last")

est_payoff_mat <- xtabs(expected_payoff ~ hand1 + hand2, data=post_estimation)
est_payoff_mat
```

hand2

hand1	1	2	3	4	5
1	0.5067220	0.3850426	0.6188228	0.2954541	0.5126862
2	0.6298342	0.5582412	0.5353389	0.3591482	0.4064244
3	0.4295398	0.4991332	0.5001834	0.6274470	0.5990136
4	0.7479080	0.6027512	0.3889548	0.4862497	0.2950853
5	0.5107168	0.6048418	0.3794254	0.7416762	0.4864391

Recovering parameters

If the probability of winning is actually a combination of difference in abilities and advantage of the hand played, can we recover these parameters?

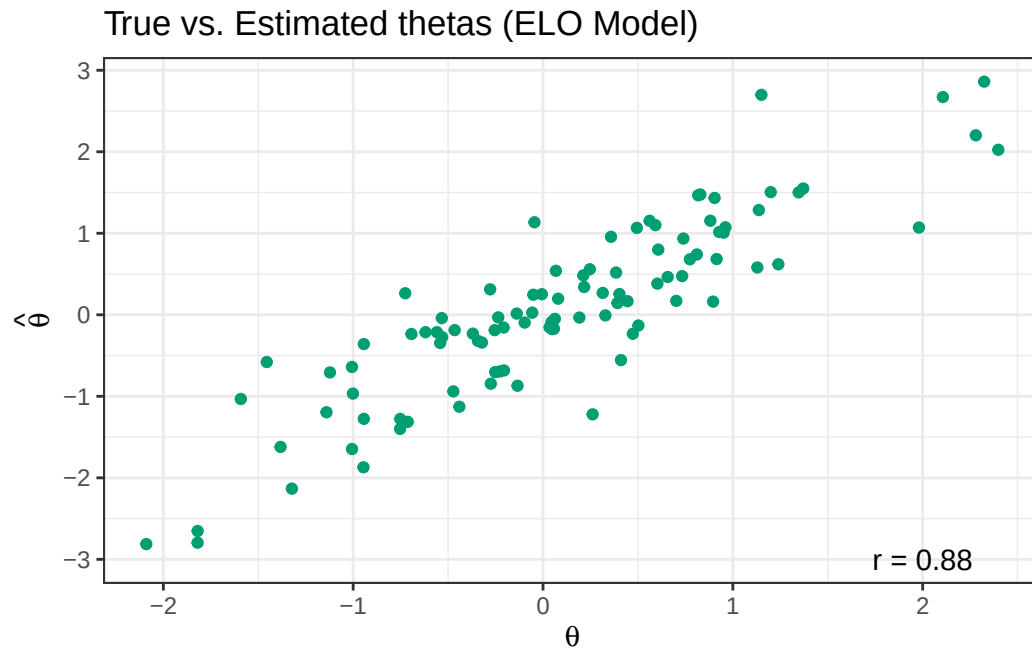
```
N_ids <- 100
N_games <- 1000
N_opts <- 5

NT_thetas <- rnorm(N_ids)
delta_phi <- 2*pi/N_opts
NT_phis <- seq(from=0,to=(2*pi-delta_phi),length.out=N_opts)

NT_matches <- as.data.frame(do.call(
  rbind,
  replicate(N_games,simulate_NT_model(NT_thetas,NT_phis,N_opts,TRUE),simplify = FALSE)))

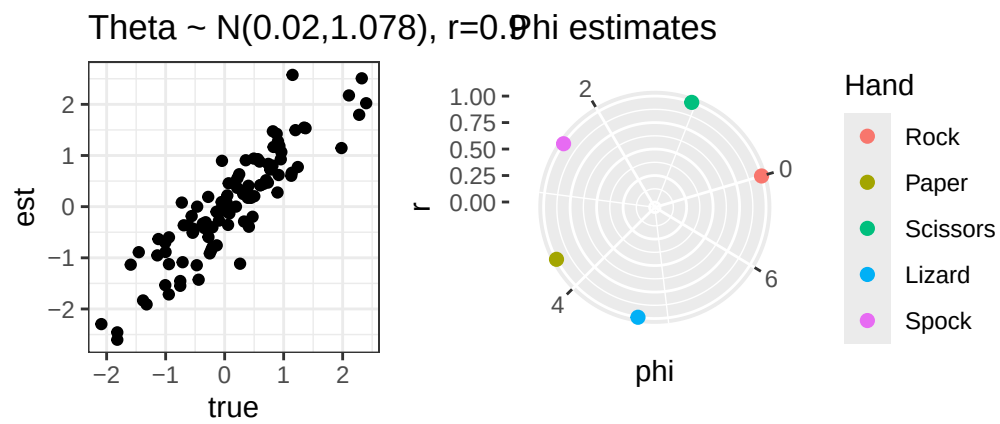
NT_matches <- NT_matches |> rename(player1="V1",
                                   player2="V2",
                                   hand1 = "V3",
                                   hand2 = "V4",
                                   result="V5")

sol <- fit_elo_model(N_ids,NT_matches)
plot_elo(sol,NT_thetas)
```

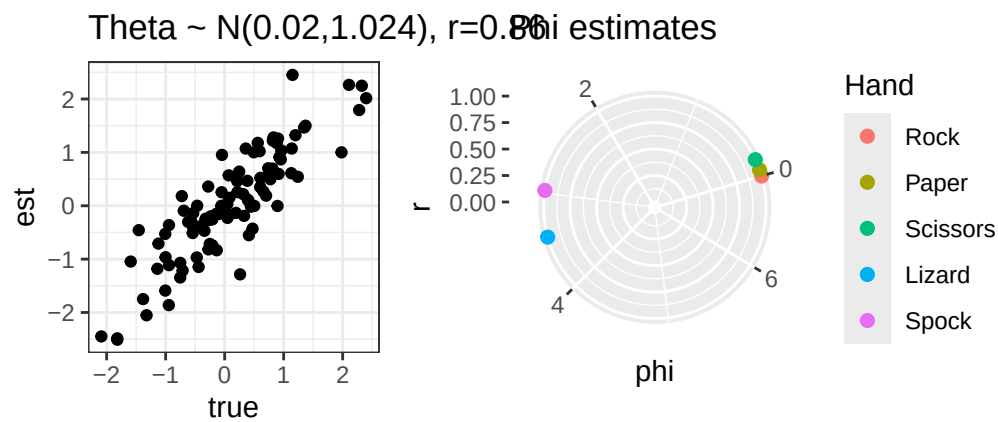
```
res <- estimate_nt_elo(N_ids,N_opts,NT_matches,10)
sol <- res$sol
plot_nt_elo(sol,NT_thetas,N_ids,N_opts,labels=c("Rock","Paper","Scissors","Lizard","Spock"))
```

Ignoring unknown labels:
 * shape : ""



```
res <- estimate_nt_elo_experiemment(N_ids,N_opts,NT_matches,10)
sol <- res$sol
plot_nt_elo(sol,NT_thetas,N_ids,N_opts,labels=c("Rock","Paper","Scissors","Lizard","Spock"))
```

Ignoring unknown labels:
 * shape : ""



Starcraft II

```
## Python code for getting Starcraft data
# key = "KzTk0G5PJS1HC6uFBtt6"
# next_url = f"/api/v1/match/?apikey={key}&format=json&limit=0&date__gte=2026-01-01&order_by="
# api_url = f"https://aligulac.com"
# games = []
# while next_url:
#     print(next_url)
#     res = requests.get(api_url+next_url).json()
#     games.extend(res.get('objects'))
#     next_url = res.get('meta').get('next')
# lotv = pd.DataFrame(games).drop(columns=['event','eventobj','offline','resource_uri','treat
# lotv['pla'] = lotv.pla.apply(lambda x: x['id'])
# lotv['plb'] = lotv.plb.apply(lambda x: x['id'])
# lotv['rta'] = lotv.rta.apply(lambda x: x['rating'])
# lotv['rtb'] = lotv.rtb.apply(lambda x: x['rating'])
# lotv['win'] = [int(a>b) for a,b in zip(lotv['sca'],lotv['scb'])]
# lotv = lotv.rename(columns={
#     'pla':'player1',
#     'plb':'player2',
```

```
#   'rca':'hand1',
#   'rcb':'hand2',
#   'win':'result',
# })
# mapping = {
#   "T":1,
#   "Z":2,
#   "P":3,
#   "R":4
# }
# lotv.hand1 = lotv.hand1.map(mapping)
# lotv.hand2 = lotv.hand2.map(mapping)
# lotv.to_csv('~Downloads/lotv_2026.csv',index=False)
```

```
starcraft <- read.csv("lotv_2026.csv")

starcraft <- starcraft |> filter(hand1!=4,hand2!=4,hand1!=hand2)

N_ids_old <- 1e6
N_ids <- 0
while (N_ids != N_ids_old){
  N_ids_old <- N_ids
  player_counts <- bind_rows(
    starcraft |> select(player = player1),
    starcraft |> select(player = player2)
  ) |>
  count(player, name = "games_played")

  # Keep only players with at least 10 games
  valid_players <- player_counts |>
    filter(games_played >= 10) |>
    pull(player)

  # Filter matches where BOTH players are valid
  starcraft_filtered <- starcraft |>
    filter(player1 %in% valid_players,
           player2 %in% valid_players)

  ids <- unique(sort(c(starcraft_filtered$player1,starcraft_filtered$player2)))
  N_ids <- length(ids)
  N_opts <- length(unique(c(starcraft_filtered$hand1,starcraft_filtered$hand2)))
  starcraft <- starcraft_filtered
```

```
}
```

```
starcraft_filtered <- starcraft_filtered|> mutate(player1 = match(player1,ids),  
          player2 = match(player2,ids))
```

```
avg_player_ratings <- rbind(starcraft_filtered |> select(player1,rta) |> group_by(player1) |>  
  starcraft_filtered |> select(player2,rtb) |> group_by(player2) |> summarise(theta=mean(rta-rtb)))
```

```
res <- estimate_nt_elo(N_ids, N_opts, starcraft,15)  
sol <- res$sol  
starting_phi <- res$starting_phi  
plot_nt_elo(sol,avg_player_ratings$theta,N_ids,N_opts,labels=c("T", "Z", "P"))
```

```
starcraft |> group_by(hand1,hand2) |> summarise(m=mean(result))
```

```
res <- fit_elo_model(N_ids, starcraft)  
plot_elo(res,avg_player_ratings$theta)
```